

Tutorial 1 - Week 1

We'll be starting at the 10 minute mark

if nothing else, write `#cleancode`

Agenda

1. TA Team
2. Tutorial Overview
3. Some Resources
4. Important Dates
5. Data Science Toolbox
6. Questions?

TA Team

Abdullah Rasul

Head TA – abdullah.rasul@mail.utotonto.ca

- **PhD Candidate** - Civil Engineering
< Robotics, Comp Vision, Reinforcement Learning />
- **Robotics Engineer** - AIS
- **MASc** - Mechanical Engineering
- **TA'd:** CSC301, APS106, CIV1287, CIV1299, ...



Sina



Yazeed



Armin

How do these tutorials work?

	1st Hour - Tutorial	2nd Hour - Project Office Hours
Focus	Weekly tutorial content	The Big Project
What we'll do	Cover the material posted on Quercus	Work on your project and get help from the TAs
Purpose	Reinforce lecture concepts & prepare for assignments	Discuss project progress, deliverables, challenges & next steps
Format	In person - code along with us!	In person - bring your project, questions & code
TAs	Led by Tutorial TA	All TAs available

Course Office Hours **are separate** → for lectures, tutorials, assignments & general course questions.

Tutorial Schedule – Fall 2026

Tutorial	Week	Topics	TA
Tutorial 1	Week 1	Data Science Toolbox	Abdullah
Tutorial 2	Week 2	Pandas	Abdullah
Tutorial 3	Week 3	HTML, Web Scraping, and Date Times	Yazeed
Tutorial 4	Week 4	JSON Files, Text Analysis, Datetimes, and Visualization	Yazeed
Tutorial 5	Week 5	Exploratory Data Analysis and Introduction to Time Series	Sina
Tutorial 6	Week 6	Geospatial Analysis	Yazeed
Tutorial 7	Week 7	Linear Regression	Sina
Tutorial 8	Week 8	Classification	Sina
Tutorial 9	Week 9	Unsupervised Learning: PCA and K-means	Sina
Tutorial 10	Week 10	End-to-End ML Pipelines	Abdullah
Tutorial 11	Week 11	Introduction to SQL	Abdullah

Some Resources

- APS106: Introduction to Computer Programming
 - Week 1: Intro, Variables
 - Week 2: Functions, Importing Modules
 - Week 3: Conditionals and Comparisons
 - Week 7: Lists, Indexing, Slicing
- Each lecture includes lecture slides, a starter notebook and a completed notebook!
- W3 Schools
- Kaggle MicroCourses
- Pandas, Seaborn and Matplotlib Official Documentation
- Stack Overflow

Important Dates

- **The Big Project: Team Formation Checkpoint**
 - (Name, Members, Roles, Quercus Group, GitHub)
- **Due Sunday September 20 | 11:59 pm**
- Use the Piazza to help you out!

Week 2 (Sep 14–20)	Lecture 2 Content Released 1:00 am	Lecture 2 - GB244 10:10 am - 1:00 pm • Lecture 2.1 - Working with DataFrames in Pandas I • Lecture 2.2 - Working with DataFrames in Pandas II • Lecture 2.3 - Working with DataFrames in Pandas III	Course Office Hours 4:10 pm–5 pm	Tutorial - SF3203 11:10 am - 12 pm • Tutorial 2: Pandas	The Big Project: Team Formation Checkpoint (Name, Members, Roles, Quercus Group, GitHub) Due 11:59 pm
	Tutorial 2 Content Released 1:00 am	Assignment 1 - Pandas Released 1:00 pm		Tutorial 2 Solution Released 2 pm	

How does that work?

How does that work?

[Anaconda](#)

[Jupyter Notebooks](#)

[PyCharm](#)

[VSCode](#)

[UofT JupyterHub](#)

[Git & Github](#)

[Programming Environments](#)

[Pickle an ML Model](#)

How does that work? - Anaconda

[Recording](#)

[Slides](#) ↓

Download Anaconda: <https://www.anaconda.com/download> ↗

Setting up Anaconda (Windows and Mac): [Setting Up Python](#)

How does that work?

How does that work?

[Anaconda](#)

[Jupyter Notebooks](#)

[PyCharm](#)

[VSCode](#)

[UofT JupyterHub](#)

[Git & Github](#)

[Programming Environments](#)

[Pickle an ML Model](#)

How does that work? - Anaconda

[Recording](#)

[Slides](#) ↓

Download Anaconda: <https://www.anaconda.com/download> ↗

Setting up Anaconda (Windows and Mac): [Setting Up Python](#)

The Data Science Toolbox

if nothing else, write `#cleancode`

IDE – Integrated Development Environment

- A software application designed to help you **write, run, and debug code efficiently**
- You *could* write Python in a basic text editor like Notepad, but an IDE gives you useful tools such as:
 - Syntax highlighting
 - Code completion
 - Error detection & debugging
 - Code navigation & refactoring
 - Integrated terminal



PyCharm



Jupyter



Visual Studio Code

Anaconda

- **Anaconda is a Python distribution - not an IDE**
- Provides a collection of tools for **Python and data science**
- Includes:
 - Python
 - conda for **package management**
 - conda for **environment management**
 - Tools such as **JupyterLab** and **Spyder**
- Simplifies **installing packages, managing dependencies, and creating isolated environments**
- **Python + Package Manager + Environment Manager + Data Science Tools**

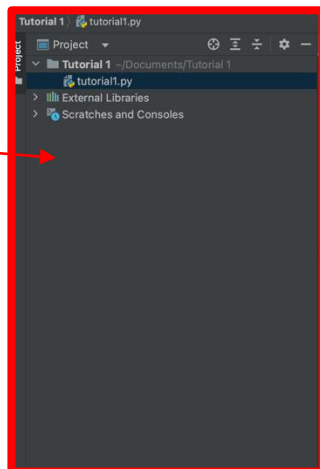


Jupyter Notebook vs. Python File

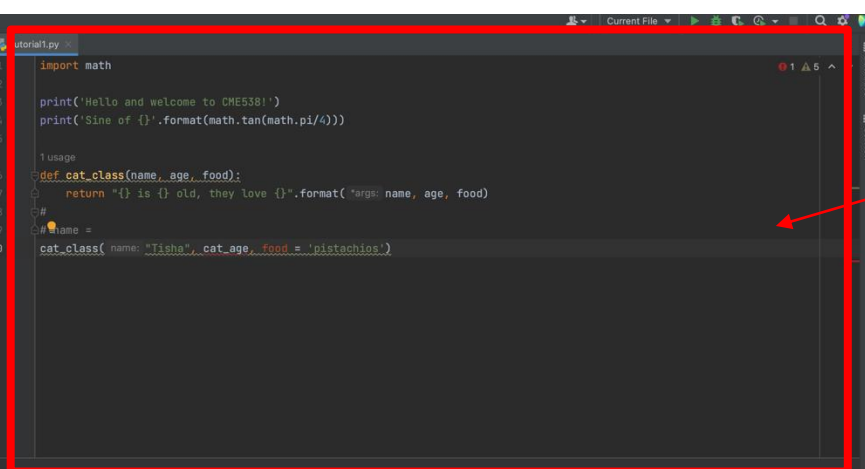
	Jupyter Notebook (.ipynb)	Python File (.py)
Structure	Organized into individual cells	A script containing Python code
Content	Mix of Python code + Markdown	Primarily Python code
Execution	Run one cell at a time	Typically runs from top to bottom
Output	Displays and saves outputs, plots, and figures inline	Output appears in the terminal/console unless explicitly saved
Best For	Exploration, visualization, learning, and explaining code	Reusable programs, automation, pipelines, and larger workflows
In CME538	Lectures and assignments use notebooks	Useful as you move toward more structured Python projects

The Anatomy of a Python File (.py) in PyCharm

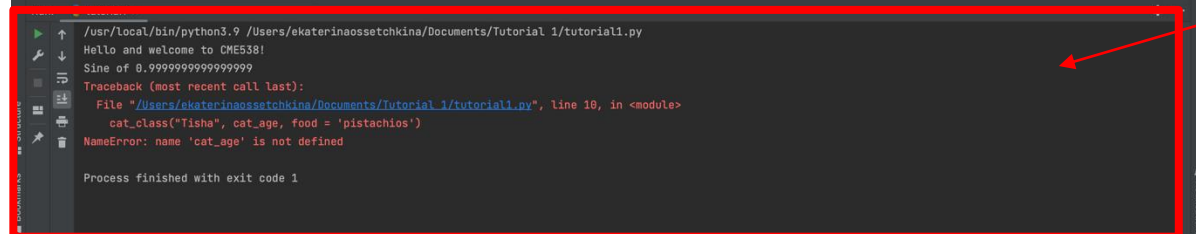
File explorer



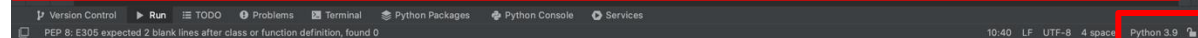
Your code!



Console (run results)



Python version
(or virtual
environment*)

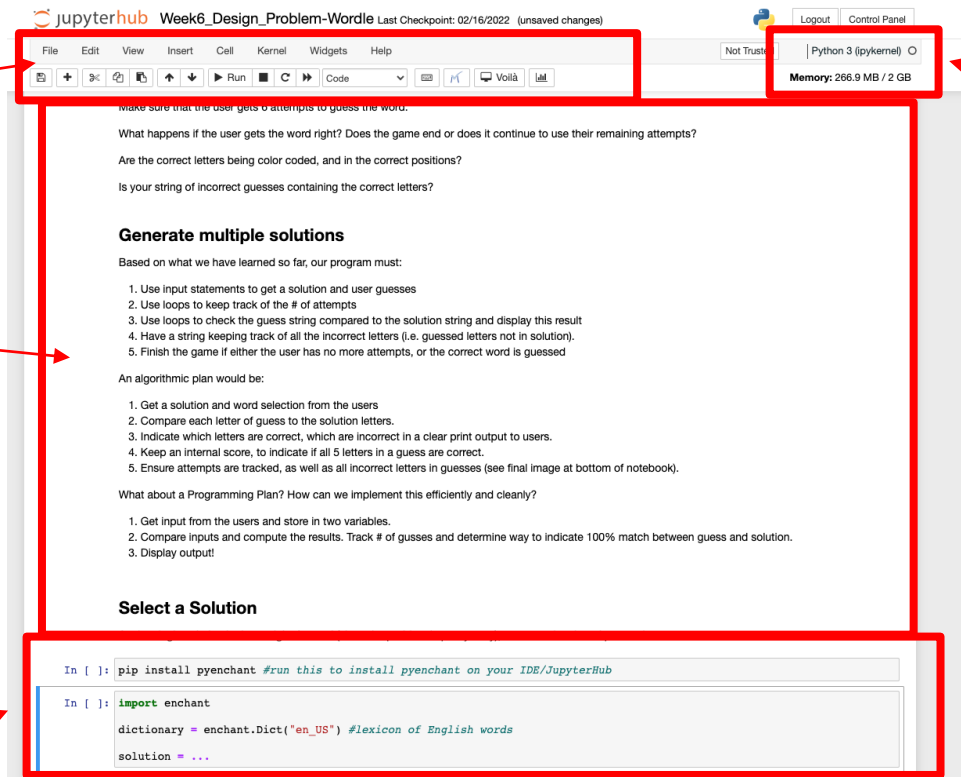


The Anatomy of a Jupyter Notebook (.ipynb) jupyter

Cell controls (run, restart, format, copy/paste...)

Markdown (.md) Text Cells

Python Code!



The screenshot shows a Jupyter Notebook interface. At the top, the header includes the Jupyter logo, the notebook title "Week6_Design_Problem-Wordle", and the last checkpoint information. Below the header is a menu bar with options like File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. A toolbar with icons for running, saving, and other actions is located below the menu bar. The main area of the notebook contains a text cell with a word game problem and a code cell with Python code. The bottom right corner shows the kernel status bar with the text "Python 3 (ipykernel)" and "Memory: 266.9 MB / 2 GB".

jupyterhub Week6_Design_Problem-Wordle Last Checkpoint: 02/16/2022 (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help

Python 3 (ipykernel) Memory: 266.9 MB / 2 GB

Make sure that the user gets 6 attempts to guess the word.

What happens if the user gets the word right? Does the game end or does it continue to use their remaining attempts?

Are the correct letters being color coded, and in the correct positions?

Is your string of incorrect guesses containing the correct letters?

Generate multiple solutions

Based on what we have learned so far, our program must:

1. Use input statements to get a solution and user guesses
2. Use loops to keep track of the # of attempts
3. Use loops to check the guess string compared to the solution string and display this result
4. Have a string keeping track of all the incorrect letters (i.e. guessed letters not in solution).
5. Finish the game if either the user has no more attempts, or the correct word is guessed

An algorithmic plan would be:

1. Get a solution and word selection from the users
2. Compare each letter of guess to the solution letters.
3. Indicate which letters are correct, which are incorrect in a clear print output to users.
4. Keep an internal score, to indicate if all 5 letters in a guess are correct.
5. Ensure attempts are tracked, as well as all incorrect letters in guesses (see final image at bottom of notebook).

What about a Programming Plan? How can we implement this efficiently and cleanly?

1. Get input from the users and store in two variables.
2. Compare inputs and compute the results. Track # of guesses and determine way to indicate 100% match between guess and solution.
3. Display output!

Select a Solution

```
In [ ]: pip install pyenchant #run this to install pyenchant on your IDE/JupyterHub
```

```
In [ ]: import enchant
```

```
dictionary = enchant.Dict("en_US") #lexicon of English words
```

```
solution = ...
```

Kernel information
(Python version +
memory)

JupyterHub

- A **cloud-based Jupyter environment** available to U of T students
- Run your notebooks **directly in your browser** - no local setup required
- Works much like running **Jupyter locally**, but your environment and files are hosted remotely
- Useful if you:
 - Have trouble setting up Python locally
 - Want a consistent environment across computers
 - Need access to **more computing resources**
- A convenient option for working on **CME538 notebooks**
- **Access:** jupyter.utoronto.ca
- Example: [Tutorial 1](#)



Demo.

if nothing else, write `#cleancode`

Questions?

if nothing else, write `#cleancode`